



Advertisement: Support JavaWorld, click here!

September 2000

HOME

FEATURED
TUTORIALS

COLUMNS

NEWS &
REVIEWS

FORUM

JW
RESOURCES

ABOUT
JW

Validation with Java and XML Schema, Part 1

Learn the value of data validation and why pure Java isn't the complete solution for handling it

Summary

If you've written many full-blown applications in Java, especially ones in which input is accepted from a Web form or a Swing GUI, you've probably spent some time writing validation code -- that is, logic for every conceivable user error. At the end of the project, though, that code was most likely thrown away, tied to the specific application. While Java provides type safety, it lacks a rich means for setting data constraints. In this series, Brett McLaughlin solves that problem; he looks at why Java alone is insufficient and examines XML Schema as a way to set up rich constraints for data used by Java programs. At the end, you'll have a reusable set of extensible components that will allow you to never code validation logic again. (2,500 words)

By Brett McLaughlin

As technologies have matured and APIs for Java and other languages have taken more of the burden of low-level coding off your hands (JMS, EJB, and XML are just a few recent examples), business logic has become more important to application coding. With this increase in business logic comes an increase in the specification of data allowed.

Read the whole "Validation with Java and XML Schema" series:

- [Part 1. Learn the value of data validation and why pure Java isn't the complete solution for handling it](#)
- [Part 2. Use XML Schema for constraining Java data](#)
- [Part 3. Parsing XML Schema to validate data](#)
- [Part 4. Build Java representations of schema constraints and apply them to Java data](#)

For example, applications no longer just accept orders for shoes; they ensure that the shoe is of a valid size, in stock, and accurately priced. The business rules that must be applied even for a simple shoe store are extremely complex. The user input and the input combination must be validated; those data often result in computed data, which may have to be validated before it is passed on to another application component. With that added complexity, you spend more time writing validation methods. You ensure that a value is a number, a decimal, a dollar amount, that it's not negative, and on, and on, and on.

With servlets and JSP pages sending all submitted parameters as textual values (an array of Java Strings, to be exact), your application must convert to a different data type at every step of user input. That converted data is most likely passed to session beans. The beans can ensure type safety (requiring an `int`, for example), but not the value range. So validation must occur again. Finally, business logic may need to be applied. (Does Doc Marten make this boot in a size 10?) Only then can computation safely be performed, and results supplied to the user. If you're starting to feel overwhelmed, good! You are starting to see the importance of validation, and why this series might be right for you.

Coarse-grained vs. fine-grained validation

The first step in making your way through the "validation maze" is breaking the validation process into two distinct parts: *coarse-grained validation* and *fine-grained validation*. I'll look at both.

Coarse-grained validation is the process of ensuring that data meet the typing criteria for further action. Here, "typing criteria" means basic data constraints such as data type, range, and allowed values. These constraints are independent of other data, and do not require access to business logic. An example of coarse-grained validation is making sure that shoe sizes are positive numbers, smaller than 20, and either whole numbers or half sizes.

Fine-grained validation is the process of applying business logic to values. It typically occurs after coarse-grained validation, and is the final step of preparation, before one either returns results to the user or passes derived values to other application components. An example of fine-grained validation is ensuring that the requested size (already in the correct format because of coarse-grained validation) is valid for the requested brand. V-Form inline skates are only available in whole sizes, so a request for a size 10 1/2 should cause an error. Because that requires interaction with some form of data store and business logic, it is fine-grained validation.

The fine-grained validation process is always application-specific and is not a reusable component, so it is beyond the scope of this series. However, coarse-grained validation can be utilized in all applications, and involves applying simple rules (data typing, range checking, and so on) to values. In this series, I will examine coarse-grained validation and supply a Java/XML-based solution for handling it.

Data: Ever present, ever problematic

If you're still not convinced of the need for this sort of utility, consider the fact that data has become *the* commodity in today's global marketplace. It is not applications, not technology, not even people that drive business -- it is raw data. The tasks of selecting a programming language, picking an application server, and building an application are all byproducts of the need to support data. Thus, those decisions may all later be revisited and changed. (Ever had to migrate from SAP or dBase to Oracle? Ever switched from NetDynamics to Lutris Enhydra?)

However, the fundamental commodity, data, never changes. Platforms change, software changes, but you never hear anyone say, "Well, let's just trash all that old customer data and start fresh." So the problem of constraining data is a fundamental one. It will always be part of any application, in *any* language. And data is always problematic because of problematic users. People type too fast, type too slow, make a silly mistake, or spill coffee on their keyboards -- the bottom line is that validation is essential to preserving accurate data, and therefore is essential to a good application. With that in mind, I'll show you how people are solving that common problem today.

Current solutions (and problems)

Since data validation is so important, you'd probably expect there to be plenty of solutions for the problem. In reality, most solutions for handling validation are clumsy and not at all reusable, and result in a lot of code applicable only in specific situations. Additionally, that code often gets

intertwined with business logic and presentation logic, causing trouble with debugging and troubleshooting. Of course, the most common solution for data validation is to ignore it, which causes exceptions for the user. Obviously, none of those are good solutions, but understanding the problems they *don't* solve can help establish requirements for the solution built here.

A big hammer

The most common way to handle data validation (besides ignoring it) is also the most heavy-handed. It involves simply coding the validation directly into the servlet, class, or EJB that deals with the data. In this example, validation is performed as soon as a parameter is obtained from a servlet:

Inline validation in a servlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ShoeServlet extends HttpServlet {

    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        // Get the shoe size
        int shoeSize;
        try {
            shoeSize = Integer.parseInt(req.getParameter("shoeSize"));
        } catch (NumberFormatException e) {
            throw new IOException("Shoe size must be a number.");
        }

        // Ensure viable shoe size
        if ((shoeSize <= 0) || (shoeSize > 20)) {
            throw new IOException("Invalid shoe size.");
        }

        // Get the brand
        String brand = req.getParameter("brand");

        // Ensure correct brand
        if (!validBrand(brand)) {
            throw new IOException("Invalid shoe brand.");
        }

        // Ensure correct size and brand
        if (!validSizeForBrand(shoeSize, brand)) {
            throw new IOException("Size not available in this brand.");
        }

        // Perform further processing

    }
}
```

This code is neither cleanly separated nor reusable. The specific parameter, `shoeSize`, was presumably obtained from a submitted HTML form. The parameter is converted to a numeric value (hopefully!), then compared to the maximum and minimum acceptable values. This example doesn't even check for half sizes. In an average case where four or more parameters are received, the servlet's validation portion alone could result in more than 100 lines of code. Now imagine increasing that to 10 or 15 servlets. This approach results in a massive amount of code, often difficult to understand and poorly documented.

In addition to the code's lack of clarity, the business logic often mixes with the validation, making code modularization very difficult. In the following example, a session bean must not only perform its business task, but also ensure that the data are correctly formatted:

Inline validation in a session bean

```
import java.rmi.RemoteException;

public class ShoeBean implements javax.ejb.SessionBean {

    public Shoe getShoe(int shoeSize, String brand) {
        // Ensure viable shoe size
        if ((shoeSize <= 0) || (shoeSize > 20)) {
            throw new RemoteException("Invalid shoe size.");
        }

        // Ensure correct brand
        if (!validBrand(brand)) {
            throw new RemoteException("Invalid shoe brand.");
        }

        // Ensure correct size and brand
        if (!validSizeForBrand(shoeSize, brand)) {
            throw new RemoteException("Size not available in this brand.");
        }

        // Perform business logic
    }
}
```

An obvious problem here is that the only way to inform the calling component of a problem is by throwing an `Exception`, usually a `java.rmi.RemoteException` in EJBs. That makes fielding the exception and responding to the user difficult, at best. Of course, each business component that uses the `shoeSize` variable must perform the same validation, which could be wedged between different blocks of business logic.

This sort of "big hammer" solution doesn't help you in reusability, code clarity, or even reporting problems to the user. This solution, the most common method for handling data validation issues, should be used only as an example of what *not* to do in your next project.

A smaller hammer

Over time, some developers have seen the "big hammer" approach's problems. As servlets' popularity has increased, handling textual parameters has been recognized as a problem worth solving. As a result, utility classes that parse parameters and convert them to a specific data type have been developed. The most popular solution is Jason Hunter's `com.oreilly.servlet.ParameterParser` class, introduced in his O'Reilly book, *Java Servlet*

Programming. (See [Resources](#).) Hunter's class allows a textual value to be supplied, formatted into a specific data type, and returned. A portion of that class is shown here:

The `com.oreilly.servlet.ParameterParser` class

```
package com.oreilly.servlet;

import java.io.*;
import javax.servlet.*;

public class ParameterParser {

    private ServletRequest req;

    public ParameterParser(ServletRequest req) {
        this.req = req;
    }

    public String getStringParameter(String name)
        throws ParameterNotFoundException {

        // Use getParameterValues() to avoid the once-deprecated
getParameter()
        String[] values = req.getParameterValues(name);
        if (values == null)
            throw new ParameterNotFoundException(name + " not found");
        else if (values[0].length() == 0)
            throw new ParameterNotFoundException(name + " was empty");
        else
            return values[0]; // ignore multiple field values
    }

    public String getStringParameter(String name, String def) {
        try { return getStringParameter(name); }
        catch (Exception e { return def; }
    }

    public int getIntParameter(String name)
        throws ParameterNotFoundException, NumberFormatException {
        return Integer.parseInt(getStringParameter(name));
    }

    public int getIntParameter(String name, int def) {
        try { return getIntParameter(name); }
        catch (Exception e) { return def; }
    }

    // Methods for other Java primitives
}
```

Two versions of the utility method are provided for each Java primitive data type. One returns the converted value or throws an exception if conversion fails, and another returns the converted value or returns a default if no conversion can occur. Using the `ParameterParser` class in a servlet

significantly reduces the problems described above:

Using the `com.oreilly.servlet.ParameterParser` class in a servlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import com.oreilly.servlet.ParameterParser;

public class ShoeServlet extends HttpServlet {

    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

        ParameterParser parser = new ParameterParser(req);

        // Get the shoe size
        int shoeSize = parser.getIntParameter("shoeSize", 0);

        // Ensure viable shoe size
        if ((shoeSize <= 0) || (shoeSize > 20)) {
            throw new IOException("Invalid shoe size.");
        }

        // Get the brand
        String brand = parser.getStringParameter("brand");

        // Ensure correct brand
        if (!validBrand(brand)) {
            throw new IOException("Invalid shoe brand.");
        }

        // Ensure correct size and brand
        if (!validSizeForBrand(shoeSize, brand)) {
            throw new IOException("Size not available in this brand.");
        }

        // Perform further processing

    }
}
```

This is a better solution, but still clumsy; you can obtain the appropriate data type, but range checking is still a manual process. It also doesn't allow, for example, just a set of values to be permitted (such as allowing only "true" or "false," rather than any textual value). Trying to implement that sort of logic in the `ParameterParser` class results in a clumsy API, with at least four different variations for each data type.

This approach also requires the acceptable values to be hard-coded into the servlet or Java class. A maximum shoe size of 20 is in the compiled code, rather than an easily changed flat file (such as a properties file or XML document). A change to that value should be trivial, but requires a code change and subsequent recompilation. This approach is a step in the right direction (kudos to

Hunter for providing the utility class), but not an answer for data validation.

Where's the toolbox?

The common problem with validation is that, in its current form, it is not reusable or compartmentalized. The `ParameterParser` class is reusable, but still requires hard-coded values and range checking. A solution that allows session beans to simply perform business logic, assuming appropriate values are supplied, does not exist. Also, there is no easy way to add functionality to the shown solutions without affecting the code -- not only the utility class itself, but the calling code too.

Additionally, these solutions are incompatible with other applications and languages. Data that do not come in a specific format (in the examples, Java Strings) cannot be plugged into the validation code. In other words, these solutions simply don't cut it for today's applications' more complex needs.

Pure Java: Not cutting it

Trying to create a solution with pure Java is a big part of the problem. Without using some sort of noncompiled format for ranges, data types, and allowed values, changes to validation rules will always result in recompilation. There are better ways to store this information; as I mentioned earlier, Java property files and XML are two formats that might help create a solution.

Property files

Java property files have been used in attempts to solve the validation problem. However, that methodology has significant flaws. First, standard Java property files do not allow multiple keys separated by periods (`key1.key2.key3 = value`). That level of nesting, while handy, is impossible without writing custom property file handling code. So a simple properties file that should look like this:

Non-standard properties file

```
field.shoeSize.minSize = 0
field.shoeSize.maxSize = 20
field.brand.allowedValue = Nike
field.brand.allowedValue = Adidas
field.brand.allowedValue = Dr. Marten
field.brand.allowedValue = V-Form
field.brand.allowedValue = Mission
```

ends up looking more like this code with a pure Java solution:

Java property files specifying validation constraints

```
shoeSizeMin = 0
shoeSizeMax = 20
```

While the key for shoe-size range becomes less clear, there is simply no way to represent the allowed values for a brand -- Java property files cannot have the same key multiple times with different values.

Some utility packages allow more advanced property file reading. (See the [Java Apache Project](#) for

an example.) However, using property files for these constraints poses a more fundamental problem: mixing basic functionality. Property files are generally used for startup parameters, configuration information, and binding names to a JNDI namespace. Mixing validation logic with those other data causes confusion, both for users and for programmers who maintain the code.

Imagine looking for the minimum shoe size allowed among properties detailing what port a Web service should start on, the recommended size of the Java heap, and on what hostname the LDAP directory server can be found. An isolated component should be used instead, just for handling validation information.

XML to the rescue?

I have examined several possible solutions for handling validation, none of which seem perfect. I propose a different approach that uses XML (and XML Schema) in concert with Java. My solution will be detailed fully in the next two articles of the series, but I'll introduce it now.

First, you can use an XML document to represent the constraints on your data. This will allow these constraints to be changed without code recompilation, simply by changing the values in the XML document. Separating constraints from other application data will also be possible. Finally, using XML and XML Schema will allow you to use a simple parser and API (which is itself a standard) to manipulate the data. No proprietary extensions or APIs are needed to handle the XML data, so the resulting code will be portable.

Here is an XML Schema that describes the previously described constraints:

Validation constraints using XML Schema

```
<?xml version="1.0"?>

<schema targetNamespace="http://www.buyShoes.com"
        xmlns="http://www.w3.org/1999/XMLSchema"
        xmlns:buyShoes="http://www.buyShoes.com"
>

    <attribute name="shoeSize">
        <simpleType baseType="integer">
            <minExclusive value="0" />
            <maxInclusive value="20" />
        </simpleType>
    </attribute>

    <attribute name="brand">
        <simpleType baseType="string">
            <enumeration value="Nike" />
            <enumeration value="Adidas" />
            <enumeration value="Dr. Marten" />
            <enumeration value="V-Form" />
            <enumeration value="Mission" />
        </simpleType>
    </attribute>

</schema>
```


All of those constraints are defined as attributes in the XML Schema, and you can express them fully and simply. If that XML Schema could perform coarse-grained validation, applications could discard the validation code described in this article and focus on business logic. I will examine that solution later in this series.

Summary

Now that I've detailed the various problems presented by pure Java solutions for validation, you might be feeling a bit down on the language. Have no fear, though. In upcoming articles, I'll let Java help solve those problems. First, though, I'll examine XML Schema more closely and look at the richer set of constraints it allows you to set on data. In fact, to data, XML Schema will start to look like Java interfaces look to code; it can provide a *data interface* for user input.

For my next article, I will prepare some XML documents that allow handling of user input. I'll then use JDOM, a Java API for manipulating XML (and XML Schema), to code utility classes around the constraints mentioned here. As a result, you'll have a good start on your reusable components for validation, using Java and XML together. In the meantime, I hope you'll think about the problems I've identified, find an application on which you can try out next month's code, and introduce yourself to JDOM (see [Resources](#)), as I'll use it heavily. See you next month! 📖

About the author

[Brett McLaughlin](#) is an Enhydra strategist at Lutris Technologies and specializes in distributed systems architecture. He is the author of [Java and XML](#), and is involved in technologies such as Java servlets, Enterprise JavaBeans technology, XML, and business-to-business applications. With Jason Hunter, he recently founded the [JDOM](#) project, which provides a simple API for manipulating XML from Java applications. McLaughlin is also an active developer on the Apache Cocoon project and the EJBoss EJB server, and a cofounder of the Apache Turbine project.

Resources

- Get the ParameterParser class:
<http://www.servlets.com/>
- "Validation with Java and XML Schema, Part 2," Brett McLaughlin (*JavaWorld*, October 2000):
<http://www.javaworld.com/javaworld/jw-10-2000/jw-1013-validation2.html>
- "Validation with Java and XML Schema, Part 3," Brett McLaughlin (*JavaWorld*, November 2000):
<http://www.javaworld.com/javaworld/jw-11-2000/jw-1110-validation3.html>
- Bone up on JDOM to prepare for the next article -- "Easy Java/XML integration with JDOM, Part 1," Jason Hunter and Brett McLaughlin (*JavaWorld*, May 2000):
<http://www.javaworld.com/javaworld/jw-05-2000/jw-0518-jdom.html>
- "Easy Java/XML integration with JDOM, Part 2," Jason Hunter and Brett McLaughlin (*JavaWorld*, July 2000):
<http://www.javaworld.com/javaworld/jw-07-2000/jw-0728-jdom2.html>
- The JDOM Website:
<http://www.jdom.org/>
- *Java Servlet Programming*, Jason Hunter (O'Reilly & Associates, November 1998):
<http://www.oreilly.com/catalog/jservlet>
- For information on the Java Apache Project:
<http://java.apache.org/>
- For more XML than you ever needed to know -- *Java and XML*, Brett McLaughlin (O'Reilly & Associates, June 2000):
<http://www.oreilly.com/catalog/javaxml>



Advertisement: Support JavaWorld, click here!

[HOME](#) | [FEATURED TUTORIALS](#) | [COLUMNS](#) | [NEWS & REVIEWS](#) | [FORUM](#) | [JW RESOURCES](#) | [ABOUT JW](#) | [FEEDBACK](#)

Copyright © 2002 JavaWorld.com, an IDG Communications company